CesiCDP

Optimisation de tournées de livraison

CVRP et CVRPTW : modélisation, métaheuristique et étude expérimentale

Réponse à l'appel à manifestation d'intérêt de l'ADEME

Récapitulatif du projet

Mots-clés · Définitions · Contexte · Problématique · Généralisation · Livrables · Pistes de solutions · Plan d'actions

Équipe projet (4 personnes) : _____

Sommaire

1. Mots-clés 3
2. Mots à définir 3
3. Contexte 6
4. Problématique 6
5. Généralisation 6
6. Livrables 7
7. Pistes de solutions 8
7.1 Modélisation formelle 8
7.2 Preuve de complexité 8
7.3 Utilisation de l'approche exacte 8
7.4 Heuristiques constructives 9
7.5 Recherche locale 9
7.6 Choix de la métaheuristique 9
7.7 Performance en Python 9
7.8 Trafic par tranche horaire (extension) 10
7.9 Validation sur VRPLIB 10
7.10 Étude expérimentale 10
8. Plan d'actions 11
8.1 Vue d'ensemble 11
8.2 Détail des phases 11
Annexe A : Travail collaboratif avec Jupyter et Git 14
Annexe B : Références bibliographiques 15

Si le sommaire n'apparaît pas, faites un clic droit dessus puis « Mettre à jour les champs ».

1. Mots-clés

Les mots-clés ci-dessous délimitent le champ du projet. Ils sont regroupés par thème pour faciliter la recherche bibliographique.

Thème	Mots-clés
Contexte	ADEME, appel à manifestation d'intérêt (AMI), mobilité multimodale intelligente, transition écologique, logistique du dernier kilomètre
Problème	Tournée de livraison, TSP, VRP, CVRP, CVRPTW, fenêtres de temps, flotte homogène, capacité, trafic par tranche horaire
Modélisation	Recherche opérationnelle, optimisation combinatoire, graphe, PLNE, variables de décision, fonction objectif, contraintes
Complexité	Classes P et NP, NP-complet, NP-difficile, réduction polynomiale
Résolution	Heuristique, métaheuristique, recherche locale, voisinage, recuit simulé, recherche tabou, ALNS
Validation	VRPLIB, BKS, gap, étude expérimentale, plan d'expérience, tests statistiques
Outils	Python, Jupyter Notebook, NumPy, Numba, PuLP, vrplib, PEP 8, Git, storytelling

2. Mots à définir

Ces définitions constituent le vocabulaire commun de l'équipe. Elles pourront être reprises, sous une forme plus formelle, dans le notebook de modélisation.

Terme	Définition
ADEME	Agence de l'Environnement et de la Maîtrise de l'Énergie : établissement public qui finance des projets de transition écologique.
AMI	Appel à Manifestation d'Intérêt : procédure par laquelle un organisme public sollicite des propositions de projets avant de les financer.
Recherche opérationnelle (RO)	Discipline qui utilise des modèles mathématiques et des algorithmes pour prendre les meilleures décisions possibles sous contraintes.
Instance	Jeu de données concret du problème : coordonnées des clients, demandes, capacité, fenêtres de temps, etc.
TSP	Travelling Salesman Problem : trouver le plus court cycle passant une seule fois par chaque ville et revenant au point de départ.

Terme	Définition
VRP	Vehicle Routing Problem : généralisation du TSP à plusieurs véhicules partant d'un dépôt commun.
CVRP	Capacitated VRP : VRP où chaque véhicule a une capacité maximale Q et chaque client une demande q_i. Niveau B attendu.
CVRPTW	CVRP with Time Windows : CVRP où chaque client doit être servi dans un intervalle $[a_i, b_i]$. Niveau A avancé.
Fenêtre de temps dure / souple	Dure : interdiction stricte de livrer hors fenêtre (cas du projet). Souple : autorisé avec pénalité. Ici, arriver en avance est permis à condition d'attendre l'ouverture.
Temps de service (s_i)	Durée passée chez le client pour effectuer la livraison.
TDVRP	Time-Dependent VRP : variante où le temps de trajet dépend de l'heure de départ (trafic par tranche horaire).
PLNE	Programme Linéaire en Nombres Entiers : modèle composé de variables de décision (dont certaines entières ou binaires), d'une fonction objectif linéaire et de contraintes linéaires.
Variable de décision	Ce que l'on choisit, par exemple : le camion va-t-il directement du client i au client j ?
Fonction objectif	Ce que l'on minimise : durée ou distance totale des tournées.
Contrainte	Règle à respecter : visite unique de chaque client, capacité, fenêtres de temps, etc.
Sous-tour	Cycle qui ne passe pas par le dépôt ; il doit être interdit dans le PLNE (contraintes MTZ ou contraintes de capacité arrondies).
P, NP, NP-complet, NP-difficile	Classes de difficulté des problèmes. Un problème NP-difficile n'admet (a priori) pas d'algorithme exact en temps polynomial.
Réduction polynomiale	Transformation d'un problème connu A en un problème B, prouvant que B est au moins aussi difficile que A.
Heuristique	Méthode rapide qui produit une bonne solution sans garantie d'optimalité (plus proche voisin, Clarke & Wright…).
Métaheuristique	Stratégie générale qui guide des heuristiques pour explorer l'espace des solutions et échapper aux optima locaux (recuit simulé, tabou, ALNS, algorithme génétique, colonies de fourmis…).
Voisinage	Ensemble des solutions obtenues en appliquant une petite modification (opérateur) à une solution : 2-opt, relocate, swap, Or-opt, 2-opt*…
Optimum local / global	Meilleure solution dans un voisinage / sur tout l'espace de recherche.

Terme	Définition
ALNS	Adaptive Large Neighborhood Search : on « détruit » une partie de la solution puis on la « répare », en adaptant les poids des opérateurs selon leurs performances passées.
Recuit simulé	Métaheuristique qui accepte parfois des solutions moins bonnes, avec une probabilité décroissant au fil d'une « température ».
VRPLIB	Bibliothèque Python (pip install vrplib) et collection d'instances de référence (Uchoa « X », Solomon, Gehring-Homberger…) avec leurs meilleures solutions connues.
BKS	Best Known Solution : meilleure solution connue pour une instance de référence.
Gap	Écart relatif à la BKS : gap = 100 × (coût obtenu − coût BKS) / coût BKS.
Plan d'expérience	Organisation rigoureuse des tests (facteurs, niveaux, répétitions) permettant des conclusions statistiquement fiables.

3. Contexte

Depuis les années 1990, la prise de conscience de la nécessité de réduire la consommation d'énergie et les émissions de gaz à effet de serre s'est imposée : protocole de Kyoto signé en 1997 (entré en vigueur en 2005), puis engagements plus ambitieux comme la division par quatre des émissions françaises d'ici 2050 ou les objectifs de grandes villes comme Paris. Les pouvoirs publics ne pouvant pas contraindre directement entreprises et particuliers, l'action porte avant tout sur l'évolution des comportements : économie et recyclage des matières premières, amélioration des modes de transport et des performances énergétiques des bâtiments.

Dans ce cadre, l'ADEME lance un appel à manifestation d'intérêt pour financer des démonstrateurs et expérimentations de nouvelles solutions de mobilité, pour les personnes comme pour les marchandises, adaptées à différents types de territoires.

CesiCDP, déjà implantée dans le domaine de la Mobilité Multimodale Intelligente, constitue une équipe de quatre personnes pour répondre à cet appel et décrocher de nouveaux marchés financés. L'étude porte sur la **gestion de tournées de livraison** : calculer, sur un réseau routier, une tournée reliant un sous-ensemble de clients puis revenant au point de départ, en minimisant la durée totale et en tenant compte du trafic prévu selon les tranches horaires. Optimiser ces tournées réduit les kilomètres parcourus, donc la consommation et les émissions ; les applications sont nombreuses (courrier, colis, ramassage des ordures, entretien du réseau routier).

Le périmètre reste à préciser : une version de base (flotte de k camions de capacité identique, instances de plusieurs milliers de villes, étude statistique) et une version avancée (fenêtres de temps strictes, avec possibilité d'attendre leur ouverture). La date limite de dépôt auprès de l'ADEME est fixée au 12 juillet.

4. Problématique

> Comment concevoir et implémenter en Python une méthode de recherche opérationnelle capable de construire, en un temps de calcul raisonnable, des tournées de livraison pour une flotte de k camions de capacité identique (et, en version avancée, sous fenêtres de temps strictes avec attente autorisée) qui minimisent la durée ou distance totale, sur des instances allant jusqu'à plusieurs milliers de clients, avec un écart inférieur à 7–10 % par rapport aux meilleures solutions connues ?

5. Généralisation

Le projet relève de l'**optimisation combinatoire**, et plus précisément de la famille des problèmes de tournées de véhicules, qui généralise le TSP. Ces problèmes sont NP-difficiles : les méthodes exactes (PLNE) deviennent inutilisables au-delà de quelques dizaines de clients. Il faut donc arbitrer entre qualité de la solution et temps de calcul, en s'appuyant sur des métaheuristiques validées expérimentalement.

La démarche suivie (modéliser, prouver la complexité, résoudre de façon approchée, valider statistiquement) est transposable à de nombreux autres problèmes : ordonnancement de production, planification de personnel, routage de réseaux, collecte de déchets ou tournées de maintenance.

6. Livrables

Livrable	Échéance	Évalué	Contenu attendu
Livrable 1 : Modélisation (Notebook Jupyter)	Au cours du prosit 3 (check)	Non	Contexte et reformulation du problème Représentation formelle des données, du problème et de l'objectif Preuve de complexité par réduction PLNE complet (variables, objectif, contraintes), sans implémentation Références bibliographiques Validation par le tuteur
Livrable 2 : Notebook final	La veille de la soutenance	Oui	Modélisation corrigée et usage de l'approche exacte Métaheuristique formalisée (solutions, voisinages, opérateurs, guidage) Implémentation, cas de test, démonstrations Étude expérimentale (≥ 20 instances, VRPLIB, gap, statistiques, limites, améliorations)
Livrable 3 : Soutenance orale (en anglais selon la feuille de route)	Dernier jour du bloc	Oui	Rappel du contexte et des objectifs Démonstration live sur de petites instances Résultats clés, difficultés, avancement et planning

Exigences transverses

- Code lisible, commenté, privilégiant la performance, conforme aux recommandations PEP 8 (pas de standard industriel exigé).
- Rédaction des notebooks orientée **storytelling** : la démarche doit se lire comme un récit cohérent.
- Niveau B : **CVRP** (capacité identique pour tous les véhicules). Niveau A : **CVRPTW** (capacité + fenêtres de temps).

Critère éliminatoire pour la note maximale

Un gap supérieur à 7–10 % par rapport aux meilleures solutions connues entraîne une note maximale de B. Les autres critères sont le temps de calcul, la qualité de l'analyse expérimentale et la pertinence des conclusions.

7. Pistes de solutions

7.1 Modélisation formelle

On travaille sur un graphe complet orienté G = (V, A) muni des données suivantes :

- V = {0, 1, …, n}, où 0 est le dépôt et C = {1, …, n} l'ensemble des clients ;
- un coût ou temps de trajet c_{ij} pour chaque arc (i, j) ∈ A ;
- une demande q_i par client et une capacité Q identique pour les k camions ;
- pour le CVRPTW : une fenêtre $[a_i, b_i]$ et un temps de service s_i par client.

Variables de décision : x_{ij} = 1 si un camion emprunte l'arc (i, j), 0 sinon ; u_i = charge cumulée du camion après le client i ; t_i = heure de début de service chez le client i (CVRPTW).

Formulation PLNE du CVRP (deux indices, contraintes MTZ)

$\min Z = \Sigma_{(i,j)\in A}\, c_{ij} \cdot x_{ij}$	*minimiser le coût total*
$\Sigma_{j\in V}\, x_{ij} = 1 \qquad \forall\, i \in C$	*un départ par client*
$\Sigma_{i\in V}\, x_{ij} = 1 \qquad \forall\, j \in C$	*une arrivée par client*
$\Sigma_{j\in C}\, x_{0j} \le k$	*au plus k camions*
$\Sigma_{j\in C}\, x_{0j} = \Sigma_{i\in C}\, x_{i0}$	*retour au dépôt*
$u_j \ge u_i + q_j - Q(1 - x_{ij}) \quad \forall\, i \ne j \in C$	*capacité + anti-sous-tours*
$q_i \le u_i \le Q \qquad \forall\, i \in C$	*bornes de charge*
$x_{ij} \in \{0, 1\} \qquad \forall\, (i, j) \in A$	*intégrité*

Extension CVRPTW

$t_j \ge t_i + s_i + c_{ij} - M_{ij}(1 - x_{ij}) \quad \forall\, i \ne j$	*cohérence temporelle*
$a_i \le t_i \le b_i \qquad \forall\, i \in V$	*fenêtres de temps*

L'inégalité (et non l'égalité) modélise directement la possibilité d'attendre l'ouverture de la fenêtre. Il est préférable de prendre $M_{ij} = b_i + s_i + c_{ij} - a_j$ plutôt qu'un grand M arbitraire, afin d'obtenir un modèle plus serré. Pour aller plus loin, on peut présenter la formulation à trois indices (x_{ijk} par véhicule) et les contraintes de capacité arrondies, plus fortes mais en nombre exponentiel.

7.2 Preuve de complexité

- **CVRP** : réduction du TSP vers le CVRP. Avec k = 1 et $Q \ge \Sigma\, q_i$, le CVRP est exactement le TSP. Le TSP étant NP-difficile (sa version décision est NP-complète par réduction depuis le cycle hamiltonien, Karp 1972), le CVRP l'est aussi. Décider si k camions suffisent revient en outre à un problème de bin packing, lui aussi NP-complet.
- **CVRPTW** : il généralise le CVRP (fenêtres infinies), il est donc NP-difficile. Résultat plus fort : même trouver une solution réalisable avec une flotte fixée est NP-complet (Savelsbergh, 1985).

7.3 Utilisation de l'approche exacte

Le PLNE est résolu avec PuLP (solveur CBC) ou OR-Tools sur de petites instances (environ 10 à 20 clients). Cela permet de :

- valider la métaheuristique en la comparant à l'optimum ;
- montrer l'explosion du temps de calcul quand n augmente, ce qui justifie le recours aux métaheuristiques ;
- obtenir des bornes inférieures via la relaxation linéaire.

7.4 Heuristiques constructives

Elles fournissent la solution initiale et servent de référence (baseline) :

- plus proche voisin, puis découpage des tournées selon la capacité ;
- algorithme des économies de Clarke & Wright (savings), très bon point de départ pour le CVRP ;
- algorithme de balayage (sweep) ;
- heuristique d'insertion I1 de Solomon, adaptée aux fenêtres de temps.

7.5 Recherche locale

Opérateurs intra-route (2-opt, Or-opt) et inter-routes (relocate, swap, 2-opt*, cross-exchange). Pour traiter des milliers de clients, deux éléments sont indispensables :

- **listes de voisinage granulaires** : ne tester que les k plus proches voisins de chaque client ;
- **évaluation incrémentale (delta) en O(1)** : calculer la variation de coût sans réévaluer toute la solution.

7.6 Choix de la métaheuristique

Méthode	Principe	Intérêt pour le projet
ALNS + recuit simulé (recommandée)	Destruction (aléatoire, pire coût, Shaw, suppression de route) puis réparation (glouton, regret-k), poids adaptatifs, acceptation par recuit simulé	Très bon rapport qualité / complexité d'implémentation ; s'adapte bien au CVRPTW
Recherche locale itérée (ILS)	Alternance perturbation / recherche locale	Simple et efficace si la recherche locale est solide
Recherche tabou	Interdiction temporaire des mouvements récents	Classique et éprouvée pour le VRP (Taillard, Cordeau)
Algorithme génétique hybride (HGS)	Population + croisement + recherche locale	État de l'art (gap < 1 %), mais complexe ; à citer comme référence

7.7 Performance en Python

- matrice de distances précalculée avec NumPy ;
- boucles critiques compilées avec Numba ;

- structures de données légères (listes de routes, tableaux de charges et d'heures d'arrivée) ;
- critère d'arrêt par temps limite ou nombre d'itérations sans amélioration.

7.8 Trafic par tranche horaire (extension)

Le sujet mentionne la prise en compte du trafic, mais la grille d'évaluation et les instances VRPLIB n'en tiennent pas compte. On peut le traiter comme une extension : temps de trajet constants par morceaux selon l'heure de départ, en respectant la propriété FIFO (un véhicule partant plus tard ne peut pas arriver plus tôt ; modèle d'Ichoua et al., 2003). Ce périmètre doit être validé avec le tuteur lors du check.

7.9 Validation sur VRPLIB

Les instances sont chargées avec `vrplib.read_instance()` et les meilleures solutions connues avec `vrplib.read_solution()` :

- **CVRP** : instances X d'Uchoa et al. (100 à 1000 clients), avec positions du dépôt et répartitions des clients variées ;
- **CVRPTW** : instances de Solomon (100 clients, classes C, R et RC) et de Gehring-Homberger (200 à 1000 clients).

> **Pièges à vérifier**
> 1) Convention d'arrondi des distances : les instances X utilisent des distances arrondies à l'entier, alors que les BKS de Solomon sont souvent calculées en réel ou tronquées. 2) Objectif hiérarchique des BKS de Solomon : elles minimisent souvent d'abord le nombre de véhicules, puis la distance. 3) Dans les instances X, le nombre de véhicules n'est pas fixé ; le minimum est $\lceil \Sigma\, q_i / Q \rceil$.

7.10 Étude expérimentale

Élément	Contenu
Facteurs étudiés	Taille (50 → plusieurs milliers de clients), répartition (aléatoire, clusters, mixte), position du dépôt (central, excentré), tension de capacité, largeur des fenêtres de temps, paramètres de l'algorithme
Protocole	Au moins 20 instances ; 5 à 10 exécutions par instance avec des graines différentes (algorithme stochastique) ; jeu d'instances de réglage distinct du jeu de test
Indicateurs	Meilleur gap, gap moyen et écart-type, temps de calcul, nombre de véhicules, courbes de convergence
Analyses	Boîtes à moustaches des gaps, temps en fonction de n en échelle log-log (complexité empirique), analyse de sensibilité, tests de Wilcoxon / Friedman, intervalles de confiance
Scénarios « territoires »	Générateur d'instances maison (urbain dense, périurbain, rural) pour relier l'étude à l'enjeu de l'ADEME

8. Plan d'actions

Le plan ci-dessous suit la feuille de route du projet (modélisation, conception et implémentation, étude expérimentale, présentation). Le tableau de synthèse donne une vue d'ensemble ; chaque phase est ensuite détaillée.

8.1 Vue d'ensemble

#	Phase	Résultat attendu	Responsable suggéré
0	Organisation	Dépôt Git, environnement, rôles, planning	Toute l'équipe
1	Livrable Modélisation	Notebook validé par le tuteur (prosit 3)	Modélisation
2	Socle technique	Chargeur VRPLIB, évaluation, vérificateur de faisabilité, visualisation	Expérimentation
3	Approche exacte	PLNE PuLP sur petites instances	Modélisation
4	Heuristiques constructives	Baselines (plus proche voisin, Clarke & Wright)	Heuristiques
5	Recherche locale	Opérateurs avec évaluation delta, listes de voisins	Heuristiques
6	Métaheuristique	ALNS / ILS réglée, gap < 5 % visé	Métaheuristique
7	Extension CVRPTW	Version fenêtres de temps validée sur Solomon	Métaheuristique
8	Trafic (optionnel)	Temps de trajet dépendants de l'heure	Selon disponibilité
9	Étude expérimentale	Campagnes de tests, graphiques, statistiques	Expérimentation
10	Notebook final	Livrable complet et exécutable de bout en bout	Toute l'équipe
11	Soutenance	Support en anglais, démo, répétitions	Toute l'équipe

8.2 Détail des phases

Phase 0 : Organisation (semaine 1)

1. Créer un dépôt Git commun (GitHub ou GitLab de l'école) et un environnement Python figé (`requirements.txt` : numpy, numba, matplotlib, pandas, scipy, pulp, vrplib).
2. Adopter une convention PEP 8 (outil `black` ou `flake8`).

3. Répartir les rôles : modélisation et PLNE ; heuristiques et recherche locale ; métaheuristique et performance ; expérimentation, visualisation et rédaction.
4. Établir un planning rétroactif depuis la date de soutenance, avec des points d'équipe réguliers.

Phase 1 : Livrable Modélisation (jusqu'au check du prosit 3)

1. Rédiger le contexte et la reformulation du problème en mode storytelling.
2. Formaliser les données (ensembles, paramètres, hypothèses) et fixer le périmètre : CVRP au minimum, CVRPTW visé, trafic en option.
3. Rédiger la preuve de complexité (réduction TSP → CVRP, puis CVRP → CVRPTW).
4. Écrire le PLNE complet en expliquant chaque contrainte.
5. Ajouter la bibliographie, présenter au tuteur et intégrer ses retours.

Phase 2 : Socle technique

1. Écrire le chargeur d'instances VRPLIB et le chargement des BKS.
2. Construire la matrice de distances avec la bonne convention d'arrondi.
3. Définir la représentation d'une solution (liste de routes) et la fonction d'évaluation.
4. Écrire un vérificateur de faisabilité indépendant (visite unique, capacité, fenêtres de temps) : c'est le filet de sécurité de tout le projet.
5. Ajouter une fonction de visualisation des tournées (matplotlib) et une fonction de calcul du gap.

Phase 3 : Approche exacte

1. Implémenter le PLNE avec PuLP.
2. Le résoudre sur de petites instances (sous-ensembles de 8 à 20 clients).
3. Tracer le temps de résolution en fonction de n pour justifier les métaheuristiques.
4. Conserver ces optimums comme référence de validation.

Phase 4 : Heuristiques constructives

1. Implémenter le plus proche voisin et Clarke & Wright (et Solomon I1 pour le CVRPTW).
2. Mesurer leur gap sur quelques instances X : ce sont les baselines.

Phase 5 : Recherche locale

1. Implémenter relocate, swap, 2-opt et 2-opt* avec évaluation delta.
2. Ajouter les listes de voisins granulaires.
3. Accélérer avec Numba et tester la montée en charge (1000, puis 3000 clients et plus).

Phase 6 : Métaheuristique

1. Implémenter l'ALNS (ou l'ILS) : opérateurs de destruction et de réparation, poids adaptatifs, acceptation par recuit simulé, critères d'arrêt.
2. Brancher la recherche locale en phase d'intensification.
3. Régler les paramètres sur un jeu d'instances d'entraînement distinct du jeu de test, pour éviter le sur-apprentissage.

4. Viser un gap moyen inférieur à 5 % sur les instances X, pour garder de la marge sous le seuil de 7–10 %.

Phase 7 : Extension CVRPTW (niveau A)

1. Gérer les heures d'arrivée et d'attente.
2. Vérifier la faisabilité en O(1) grâce aux variables de décalage (forward slack).
3. Adapter les opérateurs aux fenêtres de temps.
4. Valider sur Solomon puis sur Gehring-Homberger.

Phase 8 (optionnelle) : Trafic par tranche horaire

Uniquement si le temps le permet et si le tuteur valide ce périmètre : temps de trajet dépendants de l'heure, testés sur des instances générées.

Phase 9 : Étude expérimentale

1. Rédiger le plan d'expérience : facteurs, niveaux, au moins 20 instances, nombre de répétitions, budget de temps.
2. Automatiser les campagnes de tests avec un script qui sauvegarde les résultats en CSV.
3. Produire les tableaux et graphiques : gap par instance, boxplots, temps en fonction de n, convergence, sensibilité aux paramètres.
4. Réaliser les tests statistiques et interpréter : forces, faiblesses, scénarios où la méthode décroche.
5. Rédiger les limites et les pistes d'amélioration, justifiées par les observations.

Phase 10 : Notebook final

1. Fusionner et corriger le notebook de modélisation.
2. Ajouter les parties résolution, implémentation et expérimentation avec un fil narratif clair.
3. Relire le code (commentaires, docstrings, PEP 8).
4. Vérifier que le notebook s'exécute de bout en bout (« Restart & Run All »).
5. Le rendre la veille de la soutenance.

Phase 11 : Soutenance (en anglais)

1. Construire le support : contexte, objectifs, modèle, méthode, démo live sur une petite instance, résultats clés (gaps, temps), difficultés, avancement et planning.
2. Préparer une démo de secours (résultats pré-calculés) en cas de problème technique.
3. Répéter à l'oral en anglais en répartissant la parole entre les quatre membres, et anticiper les questions du jury (complexité, choix de la métaheuristique, gap).

> **Conseil de priorisation**
> Sécuriser d'abord un CVRP solide (vérificateur de faisabilité, Clarke & Wright, recherche locale, ALNS, gap mesuré sur VRPLIB) avant d'attaquer les fenêtres de temps. Un niveau B propre et bien analysé vaut mieux qu'un niveau A instable.

Annexe A : Travail collaboratif avec Jupyter et Git

Git n'est pas imposé par le sujet, mais il est fortement recommandé pour une équipe de quatre : historique et sauvegarde, travail en parallèle, traçabilité des contributions et version de référence unique. On ne travaille pas dans un dossier quelconque avant d'« envoyer » les fichiers : on travaille directement dans une copie locale du dépôt, que Git synchronise avec GitHub.

Mise en place (une seule fois)

1. Une personne crée le dépôt privé sur GitHub et ajoute les trois autres membres (Settings → Collaborators).
2. Chaque membre clone le dépôt, active son environnement et lance Jupyter depuis ce dossier.
3. Optionnel mais conseillé : installer `nbstripout` pour vider automatiquement les sorties des notebooks à chaque commit.

```
$ git clone https://github.com/votre-equipe/projet-vrp.git
$ cd projet-vrp
$ jupyter notebook
$ pip3 install nbstripout && nbstripout --install
```

Cycle de travail quotidien

```
$ git pull           # récupérer le travail des autres
  ... travail dans Jupyter, enregistrement (Ctrl+S) ...
$ git add notebooks/modelisation.ipynb
$ git commit -m "Ajout de la preuve de complexité TSP -> CVRP"
$ git push           # envoyer sur GitHub
```

Si Git refuse le push parce qu'un autre membre a poussé entre-temps, faire `git pull` puis à nouveau `git push`. L'interface GitHub Desktop permet de réaliser les mêmes opérations sans terminal.

Règles d'or

- Toujours `git pull` avant de travailler, et `git push` dès qu'une étape fonctionne.
- Ne jamais modifier à deux le même notebook en même temps : un notebook = un propriétaire à un instant donné.
- Mettre le code lourd dans des fichiers `.py` importés par les notebooks (avec `%autoreload 2`) : ils se fusionnent bien mieux que les `.ipynb`.
- Faire des commits petits et fréquents, avec des messages clairs.
- Exclure les fichiers inutiles via un `.gitignore` (`.ipynb_checkpoints/`, `__pycache__/`, environnement virtuel).

Structure de dépôt proposée

```
projet-vrp/
├── vrp/              # code Python réutilisable (.py)
├── notebooks/        # livrables et brouillons
├── data/             # instances (ou script de téléchargement vrplib)
├── results/          # CSV des campagnes de tests
├── requirements.txt
└── README.md
```

Annexe B : Références bibliographiques

> **Avertissement**
> Cette liste a été produite sans accès à une base de données ni à une recherche en ligne : des erreurs sur les titres exacts, années ou revues sont possibles. Chaque référence doit être vérifiée (par exemple sur Google Scholar) avant d'être intégrée aux livrables.

1. Dantzig, G. B. & Ramser, J. H. (1959). The Truck Dispatching Problem. Management Science.
2. Miller, C. E., Tucker, A. W. & Zemlin, R. A. (1960). Integer Programming Formulation of Traveling Salesman Problems. Journal of the ACM.
3. Clarke, G. & Wright, J. W. (1964). Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. Operations Research.
4. Karp, R. M. (1972). Reducibility Among Combinatorial Problems. In Complexity of Computer Computations.
5. Lenstra, J. K. & Rinnooy Kan, A. H. G. (1981). Complexity of Vehicle Routing and Scheduling Problems. Networks.
6. Kirkpatrick, S., Gelatt, C. D. & Vecchi, M. P. (1983). Optimization by Simulated Annealing. Science.
7. Savelsbergh, M. W. P. (1985). Local Search in Routing Problems with Time Windows. Annals of Operations Research.
8. Solomon, M. M. (1987). Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. Operations Research.
9. Ichoua, S., Gendreau, M. & Potvin, J.-Y. (2003). Vehicle Dispatching with Time-Dependent Travel Times. European Journal of Operational Research.
10. Ropke, S. & Pisinger, D. (2006). An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. Transportation Science.
11. Toth, P. & Vigo, D. (2014). Vehicle Routing: Problems, Methods, and Applications (2nd ed.). SIAM.
12. Uchoa, E. et al. (2017). New Benchmark Instances for the Capacitated Vehicle Routing Problem. European Journal of Operational Research.
13. Vidal, T. (2022). Hybrid Genetic Search for the CVRP: Open-Source Implementation and SWAP* Neighborhood. Computers & Operations Research.